



ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

Fundamentos de Inteligencia Artificial

Búsquedas No informadas

José A. Montenegro

5 de febrero de 2026



Contenido

Búsqueda de Soluciones

Búsquedas No informadas

- Amplitud

- Algoritmo de Dijkstra o búsqueda de coste uniforme

- Profundidad

- Profundidad Limitada

- Profundidad Progresiva

- Búsqueda Bidireccional

- Comparativa



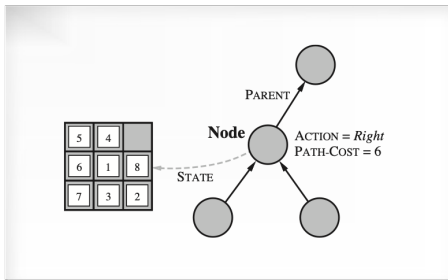
Búsqueda de Soluciones





Resolución problema

- Normalmente el espacio de estados generado por el estado inicial y la función sucesor es representado por un **árbol de búsqueda**.
- Cuando un estado es alcanzable por varios caminos tendremos un **grafo de búsqueda**.





Resolución problema

- El algoritmo general sería comenzando en la raíz, escoge, comprueba y expande hasta que encuentra una solución o no existen más estados para expandir.

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

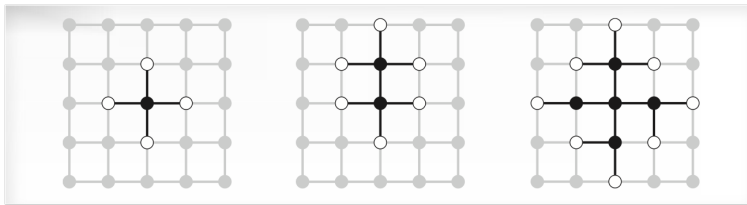
```
function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set
```



Resolución problema

Frontera : La colección (conjunto) de nodos (nodos hoja) que se generan pero no se ha expandido.

- La estrategia de búsqueda será una función que seleccione del conjunto el siguiente nodo a expandir.





Rendimiento de la resolución

Complejidad : Garantizado algoritmo encuentra solución cuando existe.

Optimización : Encuentra la estrategia la solución óptima.

Complejidad en tiempo : Cuanto tarda en encontrar la solución.

Complejidad en espacio : Cuanta memoria necesita para el funcionamiento de la búsqueda.

Eficiencia :

- Coste de la búsqueda, complejidad en tiempo y también uso memoria.
- Coste total, coste de la búsqueda y coste del camino solución encontrado.



Búsquedas No informadas





Búsquedas no informadas

- **No** tienen **información adicional** acerca de los estados más allá de la que proporciona el problema. Generan sucesores y distinguir entre un estado es objetivo o no.
- Estrategias:
 - Amplitud
 - Amplitud con costes (Dijkstra)
 - Profundidad
 - Profundidad limita
 - Profundidad progresiva
- Los problemas de búsqueda de complejidad-exponencial no pueden resolverse por métodos sin información, salvo casos pequeños.



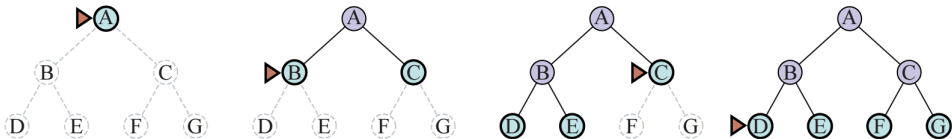
Amplitud





Amplitud (Breadth)

- Expanden todos los nodos a una profundidad en el árbol de búsqueda antes de expandir cualquier nodo del próximo nivel.
 - Encuentra una solución con un número mínimo de acciones, ya que cuando genera nodos a una profundidad d , ya ha generado todos los nodos de la profundidad $d-1$, por lo que si uno de ellos era la solución, ya la hubiera encontrado.
 - Por tanto es óptimo si todas las acciones tienen mismo coste.





Amplitud (Breadth)

- En términos de tiempo y espacio nos ponemos en el caso donde cada estado tiene b sucesores.
- La raíz de cada árbol genera b nodos, cada uno de ellos generan b nodos más, en un total de b^2 en el segundo nivel, y b^3 en el tercer nivel . . .
- Suponemos que la solución está en un nivel d , entonces el número de nodos generados es:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

- Todos los nodos se mantienen en memoria, por lo que la complejidad en tiempo y espacio son $O(b^d)$.
- Un ejemplo en el mundo real podría ser un factor $b = 10$. Si tenemos la capacidad de procesar 1 millón de nodos por segundo y cada nodo ocupa un 1Kbyte. En una profundidad de $d=10$ ($O(b^d) = O(10^{10}) = 10,000,000,000$) el procesamiento de los nodos llevaría más de 3 horas y necesitaría 10 terabytes de memoria.
- **Los requisitos de memoria son un mayor problema que el tiempo de ejecución.**



Amplitud (Breadth)

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```



Amplitud en Python

```
def breadth_first_search(problem):
    longitud_frontera = 0
    node = Node(problem.initial_state)
    if problem.is_goal(node.state):
        return node
    frontier = deque([node])
    reached = {tuple(problem.initial_state)}
    while frontier:
        node = frontier.popleft()
        print (node)
        for child_state in problem.expand(node.state):
            print (child_state)
            if tuple(child_state) not in reached:
                child_node = Node(child_state, node)
                if problem.is_goal(child_state):
                    print (f"SOL={child_state}")
                    print (f"max frontier longitud = {longitud_frontera}")
                    return child_node
                reached.add(tuple(child_state))
                frontier.append(child_node)
            if (longitud_frontera < len(frontier)):
                longitud_frontera = len(frontier)
    return "failure"
```

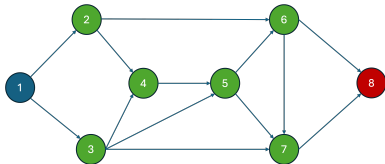


Amplitud en Python

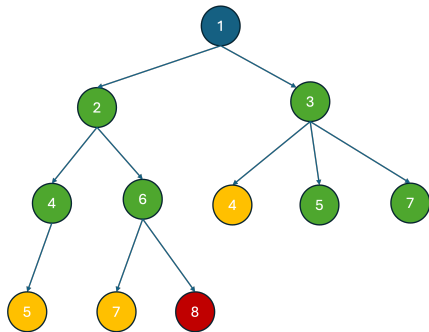
```
class Node:
    def __init__(self, state, parent=None):
        self.state = state
        self.parent = parent
    def __str__(self):
        return f"Node(state={self.state}, parent={self.parent})"
```



Ejemplo Grafo - Amplitud



- Nodo 1 - Expando nodo 1 - (2,3) Frontera: 2, 3
- Nodo 2 - Expando nodo 2 - (4,6) Frontera: 3, 4, 6
- Nodo 3 - Expando nodo 3 - (4,5,7) Frontera: 4, 6, 5, 7 (4 Repetido)
- Nodo 4 - Expando nodo 4 - (5) Frontera: 6, 5, 7 (5 Repetido)
- Nodo 6 - Expando nodo 6 - (7,8) Frontera: 5, 7, 8 (7 Repetido)
- 8 SOL





Problema Grafo en Python

```
class ProblemGrafo:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state

    def is_goal(self, state):
        return state == self.goal_state

    def expand(self, state):
        # Define cómo expandir un estado para obtener los posibles estados hijos
        sucesores = {
            '1': ['2', '3'],
            '2': ['4', '6'],
            '3': ['4', '5', '7'],
            '4': ['5'],
            '5': ['6', '7'],
            '6': ['7', '8'],
            '7': ['8']
        }
        return sucesores.get(state, []) # Devuelve los vecinos del estado o una lista vacía si el estado no tiene vecinos
```



Problema Grafo en Python

```
initial_state = '1'  
goal_state = '8'  
problem = ProblemGrafo(initial_state, goal_state)  
solution_node = breadth_first_search(problem)
```



Puzzle 8

Es un rompecabezas de fichas deslizantes. Una serie de fichas se colocan en una cuadrícula con un espacio en blanco para que el resto de las fichas puedan deslizarse en el espacio en blanco.

La formulación estándar del puzzle 8 es la siguiente:

- **Estados:** Una descripción de estado especifica la ubicación de cada una de las fichas.
 - Estado inicial: Cualquier estado puede ser designado como estado inicial.
- **Acciones:** Mientras que en el mundo físico es una ficha la que se desliza, la forma más sencilla de describir una acción es pensar que el espacio en blanco se mueve hacia la izquierda, derecha, arriba o abajo. Si el espacio en blanco está en un borde o una esquina, no todas las acciones serán aplicables.
- **Estado objetivo:** Aunque cualquier estado podría ser el objetivo, normalmente especificamos un estado con los números en orden.
- **Modelo de transición:** Asigna un estado y una acción a un estado resultante;
- **Coste de la acción:** Cada acción cuesta 1.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State



Puzzle 8 en Python

```
class Puzzle8Problem:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state

    def is_goal(self, state):
        return state == self.goal_state

    def expand(self, state):
        children = []
        zero_index = (state.index(0) // 3, state.index(0) % 3)
        moves = [(1, 0), (-1, 0), (0, 1), (0, -1)] # Movimientos: arriba, abajo, izquierda, derecha
        for move in moves:
            new_index = (zero_index[0] + move[0], zero_index[1] + move[1])
            if 0 <= new_index[0] < 3 and 0 <= new_index[1] < 3: # Verificar límites
                new_state = list(state)
                new_state[zero_index[0] * 3 + zero_index[1]], new_state[new_index[0] * 3 + new_index[1]] = \
                    new_state[new_index[0] * 3 + new_index[1]], new_state[zero_index[0] * 3 + zero_index[1]]
                children.append(new_state)
        return children
```




Ejecución Puzzle 8 en Python

```
initial_state = [1, 2 , 0, 3, 4, 5, 6, 7, 8]  
goal_state = [0, 1, 2, 3, 4, 5, 6, 7, 8]  
problem = Puzzle8Problem(initial_state, goal_state)  
solution_node = breadth_first_search(problem)
```



Imprimir Solución Problema Puzzle 8 en Python

```
def print_solution (solution_node):  
    # Reconstruir la ruta hacia la solución  
    if solution_node != "failure":  
        path = []  
        while solution_node:  
            path.append(solution_node.state)  
            solution_node = solution_node.parent  
        path.reverse()  
        print("\n\nSolución encontrada:\n")  
        for state in path:  
            #imprimirTablero(state)  
            print(state)  
    else:  
        print("No se encontró solución.")
```



Puzzle 8

Estado inicial "fácil"

```
initial_state = [1, 2, 0, 3, 4, 5, 6, 7, 8]
goal_state = [0, 1, 2, 3, 4, 5, 6, 7, 8]

Node(state=[1, 2, 0, 3, 4, 5, 6, 7, 8], parent=None)
[1, 2, 5, 3, 4, 0, 6, 7, 8]
[1, 0, 2, 3, 4, 5, 6, 7, 8]
Node(state=[1, 2, 5, 3, 4, 0, 6, 7, 8], parent=Node(state=[1, 2, 0, 3, 4, 5, 6, 7, 8], parent=None))
[1, 2, 5, 3, 4, 8, 6, 7, 0]
[1, 2, 0, 3, 4, 5, 6, 7, 8]
[1, 2, 5, 3, 0, 4, 6, 7, 8]
Node(state=[1, 0, 2, 3, 4, 5, 6, 7, 8], parent=Node(state=[1, 2, 0, 3, 4, 5, 6, 7, 8], parent=None))
[1, 4, 2, 3, 0, 5, 6, 7, 8]
[1, 2, 0, 3, 4, 5, 6, 7, 8]
[0, 1, 2, 3, 4, 5, 6, 7, 8]
SOL=[0, 1, 2, 3, 4, 5, 6, 7, 8]

max frontier longitud = 3
```



Puzzle 8

Estado inicial "fácil"

[1, 2, 0, 3, 4, 5, 6, 7, 8] **IZQ**

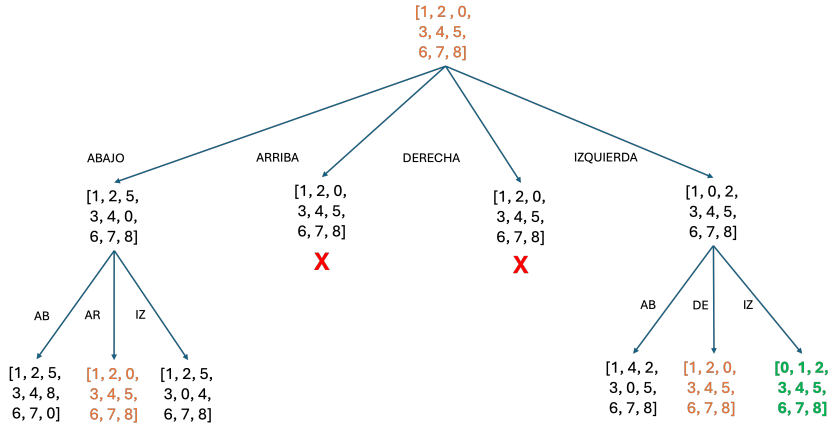
[1, 0, 2, 3, 4, 5, 6, 7, 8] **IZQ**

[0, 1, 2, 3, 4, 5, 6, 7, 8]



Puzzle 8

Estado inicial "fácil"





Puzzle 8

Estado inicial "no tan fácil"

```
initial_state = [1, 2, 3, 0, 4, 6, 7, 5, 8]  
goal_state = [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
Node(state=[1, 2, 3, 0, 4, 6, 7, 5, 8], parent=None)  
[1, 2, 3, 7, 4, 6, 0, 5, 8]  
[0, 2, 3, 1, 4, 6, 7, 5, 8]  
[1, 2, 3, 4, 0, 6, 7, 5, 8]  
...  
...  
...  
Node(state=[3, 0, 2, 4, 1, 5, 6, 7, 8], parent=Node(state=[3, 1, 2, 4, 0, 5, 6, 7, 8], parent=Node(state=[3, 1, 2, 4, 5, 0, 6, 7, 8],  
[3, 1, 2, 4, 0, 5, 6, 7, 8]  
[3, 2, 0, 4, 1, 5, 6, 7, 8]  
[0, 3, 2, 4, 1, 5, 6, 7, 8]  
Node(state=[3, 1, 2, 0, 4, 5, 6, 7, 8], parent=Node(state=[3, 1, 2, 4, 0, 5, 6, 7, 8], parent=Node(state=[3, 1, 2, 4, 5, 0, 6, 7, 8],  
[3, 1, 2, 6, 4, 5, 0, 7, 8]  
[0, 1, 2, 3, 4, 5, 6, 7, 8]  
SOL=[0, 1, 2, 3, 4, 5, 6, 7, 8]  
  
max frontier longitud = 21703
```



Puzzle 8

Estado inicial "no tan fácil"

[1, 2, 3, 0, 4, 6, 7, 5, 8] [1, 2, 3, 4, 0, 6, 7, 5, 8] [1, 2, 3, 4, 6, 0, 7, 5, 8] [1, 2, 0, 4, 6, 3, 7, 5, 8] [1, 0, 2, 4, 6, 3, 7, 5, 8]
[0, 1, 2, 4, 6, 3, 7, 5, 8] [4, 1, 2, 0, 6, 3, 7, 5, 8] [4, 1, 2, 6, 0, 3, 7, 5, 8] [4, 1, 2, 6, 3, 0, 7, 5, 8] [4, 1, 0, 6, 3, 2, 7, 5, 8]
[4, 0, 1, 6, 3, 2, 7, 5, 8] [4, 3, 1, 6, 0, 2, 7, 5, 8] [4, 3, 1, 6, 5, 2, 7, 0, 8] [4, 3, 1, 6, 5, 2, 0, 7, 8] [4, 3, 1, 0, 5, 2, 6, 7, 8]
[0, 3, 1, 4, 5, 2, 6, 7, 8] [3, 0, 1, 4, 5, 2, 6, 7, 8] [3, 1, 0, 4, 5, 2, 6, 7, 8] [3, 1, 2, 4, 5, 0, 6, 7, 8] [3, 1, 2, 4, 0, 5, 6, 7, 8]
[3, 1, 2, 0, 4, 5, 6, 7, 8] [0, 1, 2, 3, 4, 5, 6, 7, 8]



Algoritmo de Dijkstra o búsqueda de coste uniforme





Algoritmo de Dijkstra o búsqueda de coste uniforme

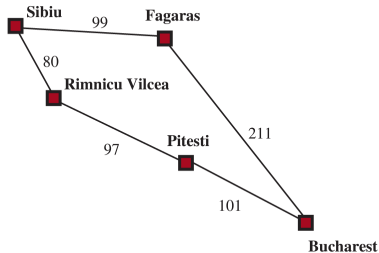
- Cuando las acciones tienen diferentes costes, una opción obvia es utilizar la búsqueda del mejor primero (veremos en el próximo tema), en la que la función de evaluación es el coste del camino desde la raíz hasta el nodo actual.
- La idea es que, mientras que la búsqueda de amplitud primero se extiende en base a una profundidad uniforme -primero profundidad 1, luego profundidad 2, y así sucesivamente-, la búsqueda de coste uniforme lo hace basándose en el coste de la trayectoria.
- Expandimos el nodo n con el coste del camino más pequeño.
 - Si son todos iguales es idéntico a la búsqueda en amplitud.
- No se preocupa por el **número de pasos** que tiene un camino, pero sí por su **coste total**.



Algoritmo de Dijkstra o búsqueda de coste uniforme

El problema es cómo ir de Sibiu a Bucharest.

- Los sucesores de Sibiu son Rimnicu Vilcea (80) y Fagaras (99).
- El siguiente nodo expandido es el nodo con menor coste, Rimnicu Vilcea, añadiendo Pitesti con coste total $80+97=177$.
- En este punto tenemos disponible Fagaras (99) y Pitesti(177) con lo cual elegimos Fagaras y añadimos Bucharest con coste 211, que hacen un total de 310.
- Bucharest es el objetivo, pero no se verifica el objetivo hasta que se expande un nodo, por lo que hay dos nodos abiertos Bucharest con 310 y Pitesti con 177, por tanto expandimos y obtenemos un coste de 278 hasta Bucharest.





Dijkstra en Python

```
class NodeCost:
    def __init__(self, state, cost, parent=None):
        self.state = state
        self.cost = cost
        self.parent = parent
    def __lt__(self, other):
        return self.cost < other.cost
    def __str__(self):
        return f"Node(state={self.state}, parent={self.parent}, cost ={self.cost})"
```



Dijkstra en Python

```
def uniform_cost_search(problem):  
    initial_node = NodeCost(problem.initial_state, 0)  
    if problem.is_goal(initial_node.state):  
        return initial_node  
  
    frontier = []  
    heapq.heappush(frontier, initial_node)  
    reached = {problem.initial_state: 0}  
  
    while frontier:  
        node = heapq.heappop(frontier)  
        if problem.is_goal(node.state):  
            return node  
  
        for child_state, step_cost in problem.expand(node.state):  
            child_cost = node.cost + step_cost  
            if child_state not in reached or child_cost < reached[child_state]:  
                child_node = NodeCost(child_state, child_cost, node)  
                reached[child_state] = child_cost  
                heapq.heappush(frontier, child_node)  
  
    return "failure"
```



Dijkstra en Python

```
class ProblemRumaniaMapa:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state

    def is_goal(self, state):
        return state == self.goal_state

    def expand(self, state):
        # Define cómo expandir un estado para obtener los posibles estados hijos y sus costos
        sucesores = {
            'Sibiu': {'Rimnicu Vilcea': 80, 'Fagaras': 99},
            'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97},
            'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
            'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101},
            'Bucharest': {'Fagaras': 211, 'Pitesti': 101}
        }
        if state in sucesores:
            return [(child_state, step_cost) for child_state, step_cost in sucesores[state].items()]
        else:
            return [] # Si el estado no tiene sucesores, retornar una lista vacía
```



Dijkstra en Python

```
# Ejemplo de uso  
initial_state = 'Sibiu'  
goal_state = 'Bucharest'  
problem = ProblemRumaniaMapa(initial_state, goal_state)  
solution_node = uniform_cost_search(problem)
```



Dijkstra en Python

```
class ProblemMalagaMapa:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state

    def is_goal(self, state):
        return state == self.goal_state

    def expand(self, state):
        sucesores = {
            'Málaga': {'Torremolinos': 13, 'Rincón de la Victoria': 15, 'Alhaurín de la Torre': 18},
            'Torremolinos': {'Málaga': 13, 'Benalmádena': 7},
            'Benalmádena': {'Torremolinos': 7, 'Fuengirola': 10},
            'Fuengirola': {'Benalmádena': 10, 'Mijas': 9},
            'Mijas': {'Fuengirola': 9, 'Coín': 18},
            'Alhaurín el Grande': {'Coín': 7, 'Cártama': 10},
            'Cártama': {'Alhaurín el Grande': 10, 'Alhaurín de la Torre': 12},
            'Alhaurín de la Torre': {'Cártama': 12, 'Málaga': 18},
        }
        if state in sucesores:
            return [(child_state, step_cost) for child_state, step_cost in sucesores[state].items()]
        else:
            return []
```



Dijkstra en Python

```
initial_state = 'Málaga'
goal_state = 'Alhaurín el Grande'

problem = ProblemMalagaMapa(initial_state, goal_state)
solution_node = uniform_cost_search(problem)

if solution_node != "failure":
    print(f"Coste solución = {solution_node.cost}")
    print("Solución encontrada:")
    current_node = solution_node
    print(current_node.state)

    while current_node.parent is not None:
        print(current_node.parent.state)
        current_node = current_node.parent
else:
    print("No se encontró solución.")
```



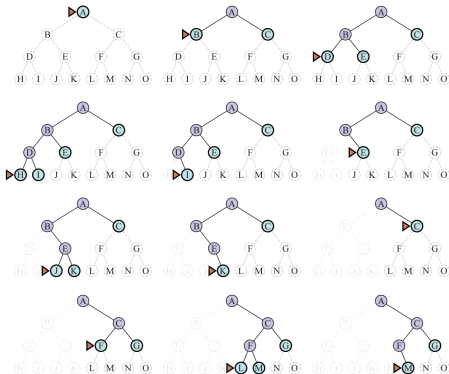

Profundidad





Profundidad (Depth)

- Expande el nodo más profundo en la frontera actual del árbol de búsqueda.



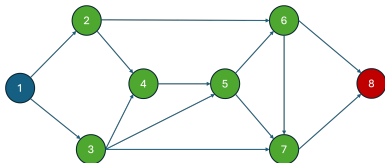


Profundidad (Depth)

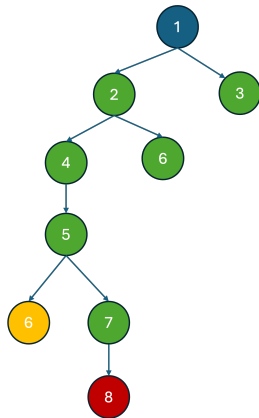
- La búsqueda procesa inmediatamente el nivel más profundo del árbol, donde los nodos no tienen sucesores. La búsqueda vuelve al siguiente nivel más profundo que todavía no expandido los sucesores.
- La búsqueda en profundidad no es óptima en coste ya que devuelve la primera solución incluso si no es la más barata.
- La ventaja que aporta este algoritmo es la necesidad de menos memoria que los algoritmos vistos hasta ahora.
- **Backtracking** es una variación que utiliza aún menos memoria, ya que solamente se genera un sucesor en vez de todos los sucesores.
- El algoritmo no es completo, ya que el algoritmo puede caer un camino infinito en un espacio de estado infinito.



Ejemplo Grafo - Profundidad



- Nodo 1 - Expando nodo 1 - (2,3) Frontera: 2, 3
- Nodo 2 - Expando nodo 2 - (4,6) Frontera: 4, 6, 3
- Nodo 4 - Expando nodo 4 - (5) Frontera: 5, 6, 3
- Nodo 5 - Expando nodo 5 - (6,7) Frontera: 7, 6, 3 (6 Repetido)
- Nodo 7 - Expando nodo 7 - (8) Frontera: 8, 6, 3
- 8 SOL





Profundidad en Python

```
def depth_first_search(problem):  
    reached = set()  
    stack = [Node(problem.initial_state)]  
  
    while stack:  
        node = stack.pop()  
        print (f"NODO SELECCIONADO {node}")  
        if problem.is_goal(node.state):  
            return node  
        if tuple(node.state) not in reached:  
            reached.add(tuple(node.state))  
            children = [Node(child_state, node) for child_state in problem.expand(node.state)]  
            stack.extend(children)  
            imprimirFrontera(stack)  
  
    return None
```



Profundidad en Python

```
initial_state = '1'  
goal_state = '8'  
problem = ProblemGrafo(initial_state, goal_state)  
solution_node = depth_first_search(problem)
```



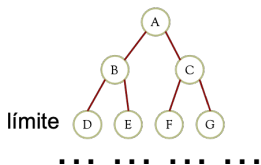
Profundidad Limitada





Profundidad Limitada

- Evitamos el problema de árboles ilimitados, estableciendo un límite de profundidad l predeterminado.
 - Nodos a profundidad l se les trata como si no tuviesen sucesor.
- Incompletitud si $l < d$, objetivo fuera alcance del límite profundidad.



 Solución



Profundidad Limitada

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node)  $>$   $\ell$  then  
      result  $\leftarrow$  cutoff  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```



Profundidad Progresiva





Profundidad Progresiva o Iterativa

- La profundidad progresiva es el método de búsqueda no informada elegido cuando el espacio de búsqueda no puede ser almacenado en la memoria y la profundidad de la solución no es conocida.

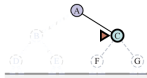
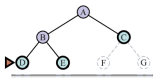
limit: 0



limit: 1



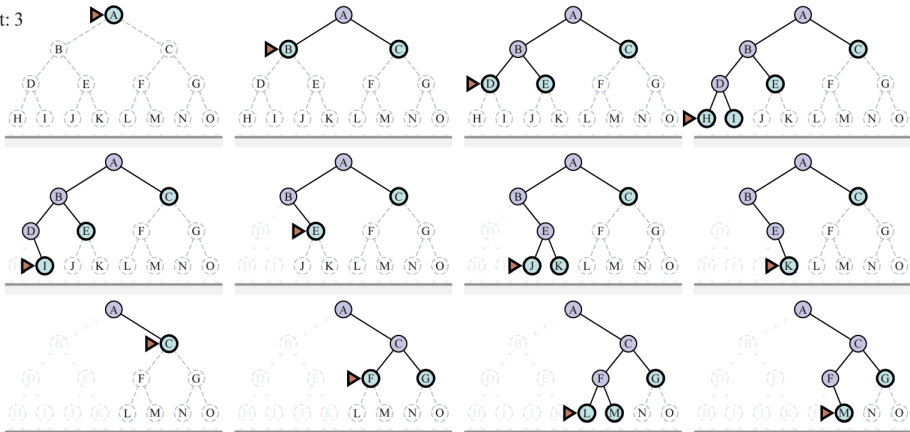
limit: 2





Profundidad Progresiva o Iterativa

limit: 3



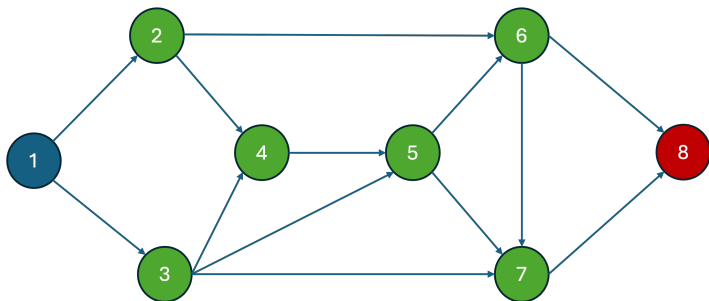


Profundidad Progresiva o Iterativa

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure  
  for depth = 0 to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```



Ejemplo Grafo - Profundidad Progresiva o Iterativa





Ejemplo Grafo - Profundidad Progresiva o Iterativa

- - **Profundidad 1**

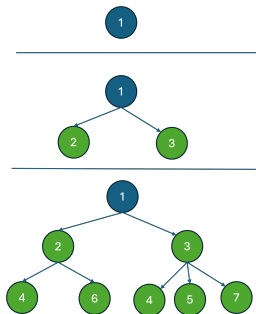
- Nodo 1. Expando nodo 1 - Fuera Limite - Frontera

- - **Profundidad 2**

- Nodo 1. Expando nodo 1 - (2,3) - Frontera 2, 3.
- Nodo 2. Expando nodo 2 - Fuera Limite - Frontera 3.
- Nodo 2. Expando nodo 3 - Fuera Limite - Frontera .

- - **Profundidad 3**

- Nodo 1. Expando nodo 1 - (2,3) - Frontera 2, 3.
- Nodo 2. Expando nodo 2 - (4,6) - Frontera 4,6,3.
- Nodo 4. Expando nodo 4 - Fuera Limite - Frontera 6,3.
- Nodo 6. Expando nodo 6 - Fuera Limite - Frontera 3.
- Nodo 3. Expando nodo 3 - (4,5,7) - Frontera 5,7. (4 Repetido)
- Nodo 5. Expando nodo 5 - Fuera Limite - Frontera 7.
- Nodo 7. Expando nodo 7 - Fuera Limite - Frontera .

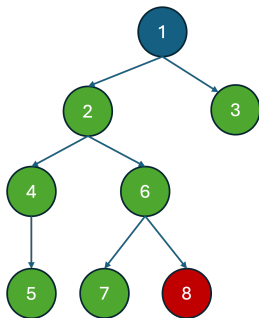




Ejemplo Grafo - Profundidad Progresiva o Iterativa

- - **Profundidad 4**

- Nodo 1. Expando nodo 1 - (2,3) - Frontera 2, 3.
- Nodo 2. Expando nodo 2 - (4,6) - Frontera 4,6,3.
- Nodo 4. Expando nodo 4 - 5 - Frontera 5,6,3.
- Nodo 6. Expando nodo 5 - Fuera Limite - Frontera 6,3.
- Nodo 3. Expando nodo 6 - (7,8) - Frontera 7,8,3.
- Nodo 5. Expando nodo 5 - Fuera Limite - Frontera 8,3.
- Nodo 8. Expando nodo 8 - Solución - Frontera 3.





Búsqueda Bidireccional





Búsqueda Bidireccional

- Realiza la búsqueda iniciando tanto desde el estado inicial como de los estados finales, con la esperanza que las búsquedas se encuentren.
- La motivación es la complejidad ya que $b^{d/2} + b^{d/2}$ es menor que b^d .
- El algoritmo mantiene dos estructuras de fronteras y de estados alcanzados. Cuando un camino en una frontera alcanza un estado que también ha sido alcanzado en la otra mitad de la búsqueda, los dos caminos son unificados para establecer una solución.
- La primera solución que obtenemos no es garantizada que sea la mejor, la función TERMINATED determinará cuando parar de buscar nuevas soluciones.



Búsqueda Bidireccional

```
function BIBF-SEARCH( $problem_F, f_F, problem_B, f_B$ ) returns a solution node, or failure  
   $node_F \leftarrow \text{NODE}(problem_F.INITIAL)$  // Node for a start state  
   $node_B \leftarrow \text{NODE}(problem_B.INITIAL)$  // Node for a goal state  
   $frontier_F \leftarrow$  a priority queue ordered by  $f_F$ , with  $node_F$  as an element  
   $frontier_B \leftarrow$  a priority queue ordered by  $f_B$ , with  $node_B$  as an element  
   $reached_F \leftarrow$  a lookup table, with one key  $node_F.STATE$  and value  $node_F$   
   $reached_B \leftarrow$  a lookup table, with one key  $node_B.STATE$  and value  $node_B$   
   $solution \leftarrow failure$   
  while not TERMINATED( $solution, frontier_F, frontier_B$ ) do  
    if  $f_F(\text{TOP}(frontier_F)) < f_B(\text{TOP}(frontier_B))$  then  
       $solution \leftarrow \text{PROCEED}(F, problem_F, frontier_F, reached_F, reached_B, solution)$   
    else  $solution \leftarrow \text{PROCEED}(B, problem_B, frontier_B, reached_B, reached_F, solution)$   
  return  $solution$ 
```

Pasamos dos versiones del problema y la función de evaluación, una en la dirección hacia delante (subíndice F) y otra hacia atrás (subíndice B).

Necesitamos llevar la cuenta de dos fronteras y dos tablas de estados alcanzados, y necesitamos ser capaces de razonar hacia atrás.



Búsqueda Bidireccional

```
function PROCEED(dir, problem, frontier, reached, reached2, solution) returns a solution
    // Expand node on frontier; check against the other frontier in reached2.
    // The variable "dir" is the direction: either F for forward or B for backward.
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
        s ← child.STATE
        if s not in reached or PATH-COST(child) < PATH-COST(reached[s]) then
            reached[s] ← child
            add child to frontier
        if s is in reached2 then
            solution2 ← JOIN-NODES(dir, child, reached2[s]))
            if PATH-COST(solution2) < PATH-COST(solution) then
                solution ← solution2
    return solution
```

si el estado s' es un sucesor de s en la dirección hacia adelante, entonces necesitamos saber que s es un sucesor de s' en la dirección hacia atrás.

Cuando un camino de una frontera alcanza un estado que también se alcanzó en la otra mitad de la búsqueda, los dos caminos se unen (mediante la función JOIN-NODES) para formar una solución.

Tenemos una solución cuando las dos fronteras chocan.



Comparativa





Comparativa

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

- b es el factor de ramificación.
- m es la máxima profundidad del árbol de búsqueda.
- d es la profundidad de la solución menos profunda, o es m cuando no hay solución
- d is the depth of the shallowest solution, or is m when there is no solution;
- ℓ es límite de profundidad.
- 1 completo si b es finito, y el espacio de estado tiene solución o es finito.e.
- 2 completo si los costes de todas las acciones $\geq \epsilon > 0$;
- 3 coste óptimo si el coste de todas las acciones son idénticas;
- 4 si ambas direcciones son aplicando algoritmo de amplitud o con coste uniforme.



That's all folks!

